

DAAB

Maturity, Roadmap & IP

Where the platform stands today, capability by capability; how much of the written specification is delivered; how it has been engineered and tested; what is still open; what is proposed next; and what the codebase depends on, with a licence audit of its third-party runtime components.



VERSION
1.0

DATE
2026-09-24

CODE BASELINE
go @ 16dc548 · python @ 6966d4b · web @
907bc8b

Contents

1 Maturity summary

Status vocabulary · capability status matrix · deployment status

2 Scope delivered vs specified

33 business-analysis specifications and 10 improvement specifications, by module

3 Development timeline

Milestones across the three code repositories · commit and velocity statistics

4 Engineering process & quality

Automated tests per repository · lint and type-check · CI/CD · practices visible in the code

5 Known limitations

Processing limits · quality gaps · hardening items before production

6 Roadmap (proposed)

Production readiness · capability depth · scale-out and breadth

7 IP & dependencies

First-party code · licence audit summary · models · external services and vendor risk

8 Appendix – licence tables

Go modules · Python packages · web packages · method

How to read this document

Every statement describes the code at the commits printed on the cover. Usage figures come from a **development/demo database snapshot taken on 2026-09-24**. They show that a capability has been exercised end-to-end with real or demo data. They are not production traffic, customer counts or accuracy measurements. Facts that cannot be derived from code (legal entity, team, pilots) are marked **TO BE COMPLETED**.

1 Maturity summary

DAAB is a knowledge-graph and data-access platform for AI agents. It turns documents and connected business databases into a queryable graph and exposes that graph to MCP clients and a web dashboard. The core pipeline (ingest, graph, search, natural-language database query, agent gateway) is implemented and has been exercised end-to-end with demo data. The platform has not yet run in production.

19

CAPABILITIES LIVE
(DEMO DATA)

8

CAPABILITIES BUILT, NO
DATA YET

8

ITEMS PLANNED /
STUBBED

Pre-prod

DEPLOYMENT STATUS

1.1 Status vocabulary

Status	Meaning in this document
LIVE	Implemented, wired and exercised end-to-end with real or demo data in the development environment. Not production usage.
BUILT	Implemented and wired (reachable through the API, MCP or dashboard), with no data yet in the snapshot.
PLANNED	On the roadmap, specified but not started, or present in code only as a stub or an unwired component.

1.2 Capability status matrix

Evidence column: exact row counts from the development/demo database snapshot, 2026-09-24.

Capability	Status	Evidence
Knowledge-graph store: typed nodes, whitelisted edges, append-only version history for graph edits	LIVE	16,509 nodes · 35,431 edges · 9,973 node versions
Document ingestion: upload, PDF text layer + OCR, decomposition into sections and chunks	LIVE	267 uploaded files · 299 canonical documents · 1,468 sections · 4,978 chunks · 473 drafts (468 processed)
Local multilingual embeddings; full-text, semantic and hybrid (RRF) search	LIVE	12,039 node embeddings (384-dim)
Cross-document retrieval (chunk-similarity linking + graph expansion)	LIVE	20,173 chunk-similarity edges
Entity extraction (typed entities and relations with provenance)	LIVE	1,743 organisations · 1,015 locations · 693 persons · 580 artefacts · 268 events
Entity resolution: candidate blocking, rules, cross-encoder filter, LLM verification, reversible merge	LIVE	10,903 merge suggestions; merges applied in this environment (apply mode is a configuration setting)
Database connectors and schema extraction: PostgreSQL, MySQL, MariaDB, SQL Server	LIVE	9 registered data sources, 5 active (2 MySQL, 1 MariaDB, 1 SQL Server, 1 PostgreSQL) · 852 tables · 7,844 columns · 943 foreign keys in the active sources
Schema-to-graph generation (foreign-key and inferred relationships)	LIVE	5 generation jobs · 663 foreign-key edges · 20 sync jobs
Natural-language query over connected databases. Agent/API path: the LLM returns a query plan that the server validates and renders as parameterised, read-only SQL. Dashboard agentic chat: the model writes SELECT statements, checked by a read-only validator with row caps	LIVE	6,280 query requests · 1,073 cached query plans
Conversational chat with streamed rich responses and tool steps	LIVE	7 threads · 48 messages

Capability	Status	Evidence
Document management system sync (pull analysed documents, incremental, per-document state)	LIVE	206 synced documents · 3 source connections
Derived-record write-back from connected systems	LIVE	3 derived records (small volume)
Gated database write path (opt-in single-row insert, approved multi-step playbooks, full audit)	LIVE	1 approved playbook · 6 audited write attempts (small volume)
MCP agent gateway: 53 MCP tools wired to the API (55 advertised; 2 without a server route, see PLANNED), stdio and Streamable HTTP, per-caller key passthrough. Two of the 53 are local code-indexing tools that need the indexer installed on the agent's host	LIVE	Running in the development stack and used by MCP clients; 80 API keys issued (16 active)
Accounts, projects, per-project roles, scoped API keys	LIVE	2 user accounts · 10 projects · 7 memberships · 80 API keys issued (16 active)
AI provider abstraction: priority failover, circuit breaker, per-function model routing, cost tracking	LIVE	4 providers · 3 registered models assigned across 8 functions · 8,649 usage-log rows
Web dashboard (33 page routes): 3D graph explorer, schema graph, ingestion workspace, admin	LIVE	Used against the dataset above
Audit trail (node history, admin actions, query submissions, write attempts)	LIVE	4,995 audit rows
Quality gates on every graph write (schema, completeness, edge whitelist)	LIVE	Applied to all writes above
Human-authored project knowledge (decisions, tasks, requirements, discoveries) via typed MCP tools	BUILT	0 nodes of these types
Code indexing: tree-sitter parsers for TypeScript/JavaScript, Python, Go, Dart, Markdown	BUILT	0 code-symbol nodes
Embedding-based table selection for NL query ("smart context")	BUILT	0 table embeddings (needs an external embedding endpoint)
NL-query benchmark suite (questions, runs, baselines)	BUILT	0 rows; dashboard page hidden
Agent sessions	BUILT	0 sessions
Agent memory of record (remember / recall, hybrid ranking)	BUILT	3 memory rows · 0 memory embeddings
OAuth-linked LLM subscription account	BUILT	0 stored tokens
Agent tool-call logging	BUILT	0 rows
In-application cloud secrets-manager provider	PLANNED	Stub; cloud deployments inject secrets as environment variables instead
Managed cloud queue (SQS) publisher	PLANNED	Stub; Redis is the working queue
Two MCP tools (context summary, impact analysis)	PLANNED	Advertised by the gateway; no server route exists yet
Legacy Python worker handlers (schema-to-graph, NL query, benchmark) and one AI-task queue	PLANNED	Present and unit-tested but not wired; superseded by the Go implementations
LLM-based SQL verification step	PLANNED	Component exists; not wired into the pipeline
API-wide rate limiting in the core server	PLANNED	Service exists; not wired. The agent gateway and the public ingestion endpoint already rate-limit.
MongoDB connector	PLANNED	Specified (improvement spec), not started
Graph community detection for retrieval	PLANNED	In the retrieval specification; not implemented

1.3 Deployment status

Environment	State
Self-hosted release v1.0.0 (2026-06-24)	BUILT Three pinned container images (API + gateway, Python workers, dashboard) with an AWS-free, production-like configuration profile. Smoke-tested at release. The code has advanced since: the baseline schema is at migration 87; the v1.0.0 smoke test migrated to version 66.
Development stack	LIVE Docker Compose, built from source, used for development and demonstrations. This is the source of the snapshot figures above.
Cloud dev environment (AWS)	BUILT Container services on ECS with managed PostgreSQL (RDS) and Redis (ElastiCache). Infrastructure-as-code modules (staging and production profiles) and CI/CD exist for the Go service; the Python and dashboard services and the managed Redis are provisioned outside the repository's infrastructure code.
Production	PLANNED No production tenant. No SLA, uptime or scale figures are claimed.
Pilots / customers	TO BE COMPLETED

2 Scope delivered vs specified

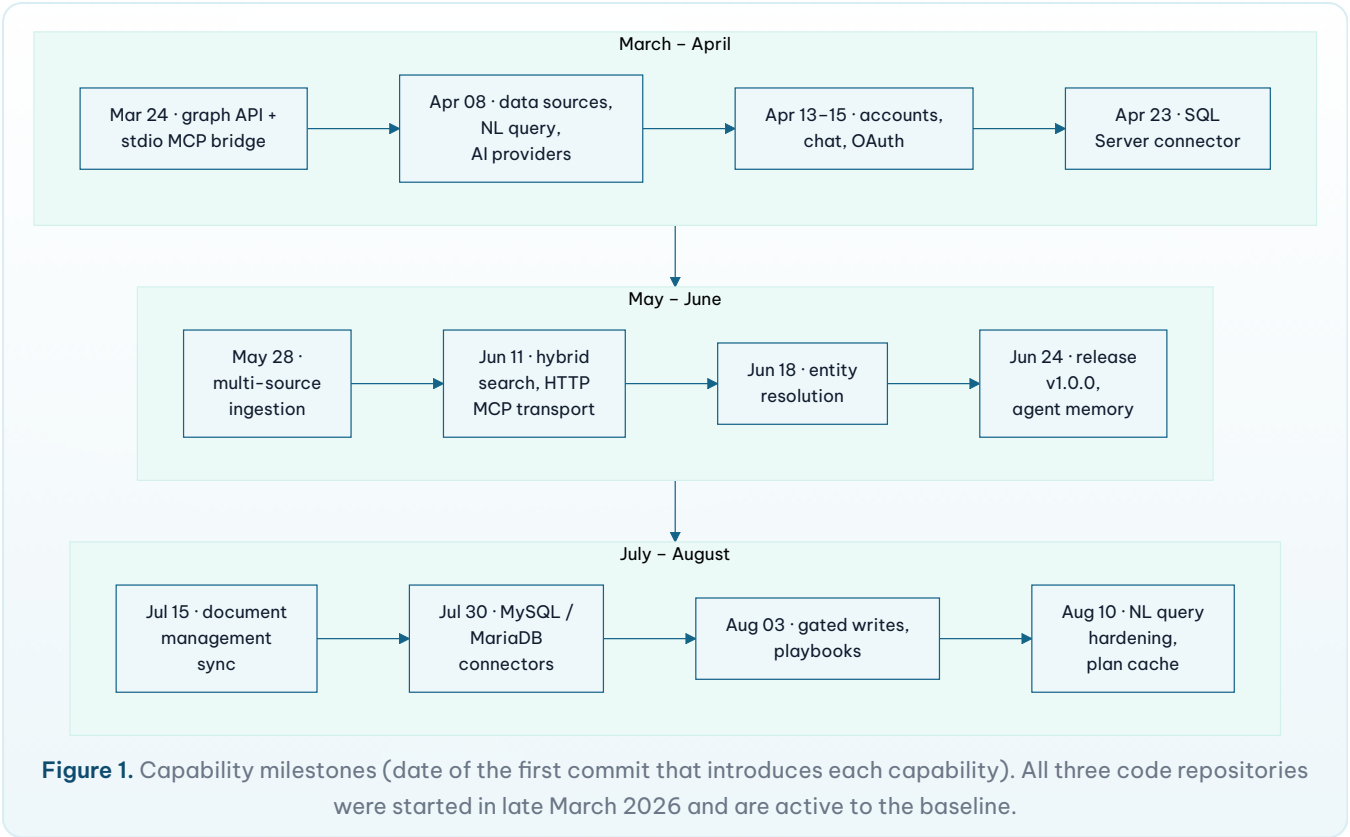
The requirements repository holds **33 business-analysis specifications** (BA-001 to BA-033) in eight phases and **10 improvement specifications** (IMP-001 to IMP-010). The business-analysis specifications have use cases, numbered business rules and non-functional requirements, and together define **249 numbered NFRs**. The improvement specifications are lighter: numbered business rules, no numbered use cases. The status headers inside the documents are mostly still "Draft" and are not maintained. The status below is derived from the code and the snapshot, using the vocabulary in §1.1. **Of the 43 specifications, 31 are delivered and exercised with data (two partially), 11 are built but not yet exercised, and 1 is not started.**

Phase	Spec	Module	Status	Note
1	Secret management	Implement the in-application cloud secrets provider (a stub today) so the provisioned parameter store and configuration bucket are actually read, supply the encryption key through infrastructure-as-code, and move it to a managed key service held by as few services as possible.		
1	BA-002	MCP bridge (agent integration)	LIVE	53 MCP tools wired to the API; 2 more await server routes
1	BA-003	Code indexing engine	BUILT	No code nodes in snapshot
1	BA-004 · 005	Web dashboard · enforcement (quality gates)	LIVE	33 page routes; schema, completeness, whitelist gates
1	BA-006	Deployment & operations	BUILT	Self-hosted release; cloud dev env; CI/CD for Go only; production PLANNED
2	BA-007-012	Data-source connection & schema extraction; graph generation from schemas; AI provider abstraction; interactive graph visualisation; natural-language query; admin sync portal	LIVE	4 database engines; failover; 3D explorer
2	BA-013	Benchmark suite	BUILT	No runs yet
3	BA-014-016	User accounts; project access control; platform administration	LIVE	Authorization hardening open (§5)

Phase	Spec	Module	Status	Note
4	BA-017-019	Conversational AI; rich response rendering; tools, actions & insights	LIVE	
5	BA-020-021	Smart context for NL→SQL accuracy · OAuth integration for an LLM subscription	BUILT	Table embeddings not populated
6	Internal service authentication	Validate credentials on the internal Python endpoints (chat, embeddings, admin) instead of relying on network placement.		
7	BA-026-027	Standalone code indexer (package + CLI); bridge serve loop & local indexing tools	BUILT	Serve loop in use; indexing unused
7	BA-028	Satellite memory ingestion & semantic recall	BUILT	3 memory rows
8	BA-029-030	Streamable HTTP + auth transport (MCP) · canonical ingest core	LIVE	299 canonical documents
8	BA-031	Graph extraction & entity resolution (2-pass)	LIVE	Started through the API; no UI trigger yet
8	BA-032	Document navigation & cross-source links	LIVE	Outline, backlinks, section neighbours
8	BA-033	Cross-document GraphRAG retrieval	LIVE	Partial: community detection PLANNED
IMP	003	MongoDB provider	PLANNED	Not started
IMP	001 · 002 · 005-007 · 010	SQL Server and MySQL/MariaDB providers · hybrid search (RRF) + multilingual embedding · OpenAI-compatible providers with per-function models · chunk retrieval layer · derived-record write-back	LIVE	010: superseded in part; small volume
IMP	004 · 008 · 009	Manual AI description generation · read-only small-model tool profile · MCP file ingest & linkage	BUILT	

3 Development timeline

The platform lives in three code repositories: the Go API and MCP gateway, the Python workers and the web dashboard. A fourth repository holds the specifications. Figure 1 shows the dates of the first commits that introduce each capability. Release v1.0.0 is documented in the release notes; it has no git tag.



1,252

COMMITTS, 3 CODE REPOS

176

CALENDAR DAYS (MAR 24 → SEP 16)

≈50

COMMITTS / WEEK, AVERAGE

3

AUTHOR IDENTITIES (ALIASES MERGED)

3.1 Commits per month

Repository	Mar	Apr	May	Jun	Jul	Aug	Sep*	Total
Go API + MCP gateway	8	186	24	335	70	65	6	694
Python workers	7	43	39	193	67	0	3	352
Web dashboard	9	91	36	27	18	4	21	206
All	24	320	99	555	155	69	30	1,252

* September counts run to 2026-09-16, the last commit in the baseline. Commit counts include merge commits.

3.2 Reading the timeline

- Two build peaks. April delivered phases 2–5: data sources, NL query, accounts and chat. June delivered phases 6–8 (ingestion, entity resolution, GraphRAG, HTTP transport) and ended with release v1.0.0.
- Post-release work (July to September) is lower in volume. It covered new connectors (MySQL/MariaDB), document management system sync, the gated write path, and hardening of NL-to-SQL accuracy. Commit volume has fallen since July.
- Specifications and code. Specifications for phases 2 onward were written alongside or ahead of their code. The phase-1 specifications were written after the phase-1 code, to document it.
- Team. Git history shows 3 distinct author identities after alias merging. That is an identity count, not headcount. Team size and roles: **TO BE COMPLETED**.

4 Engineering process & quality

4.1 Automated tests and static checks, per repository

Repository	Source size	Automated tests	Result at baseline	Lint / types
Go API + gateway	≈76,900 LOC (non-test), 22 internal packages + 3 commands	2,753 test functions in 333 test files, mostly table-driven; 17 integration-tagged files run against real PostgreSQL + pgvector; contract tests hold the MCP tool schemas and routes in step	All packages pass (<code>go test ./...</code>). Tests that need a database skip when none is reachable.	golangci-lint v2 (errcheck, ineffassign, staticcheck, unused)
Python workers	≈18,700 LOC (service + indexer package)	943 tests collected (798 service, 145 indexer; 915 <code>test_</code> functions); 141 test files with more test code than source code; 19 end-to-end tests need the full stack	922 passed, 1 failed, 20 skipped (19 end-to-end tests that need the full stack, 1 benchmark test that needs a configured project). The failure is one extraction-parser unit test.	Ruff: 16 findings (2 in source, 14 in tests), 15 auto-fixable. Coverage measurement is not configured.
Web dashboard	≈30,200 LOC TypeScript (strict)	No automated tests yet: no unit, component or end-to-end suite	TypeScript type-check passes	ESLint reports 27 errors and 28 warnings; no step enforces it

Observed 2026-09-24 on local working trees that match the baseline except for three small uncommitted edits (two Go configuration files, one dashboard component). The Go run excluded integration-tagged files. "Test functions" counts `func Test...` declarations, each of which may hold many sub-cases.

4.2 Continuous integration and delivery

Repository	Pipeline
Go API + gateway	CI on every push and pull request: lint, <code>go mod tidy</code> drift check, unit tests with the race detector and a coverage profile, a cross-project isolation integration gate against PostgreSQL + pgvector, and an image build. A coverage percentage is not yet reported.
Go deploy: staging	On every push to main: build the Go API image, push it to the staging container registry, roll the Go API service of the cloud dev environment onto it, publish the configuration files to S3 and run a health check.
Go deploy: production	Started manually or by a published release. It requires the image to already exist in the staging registry, promotes it to a separate production registry, performs a rolling update of the production Go API service with a stability wait, publishes the production configuration, and runs a health check; the ECS deployment circuit breaker rolls back a failed rollout, and the workflow also has a rollback job. The workflow is BUILT ; there is no production environment yet (§1.3). Both deploy workflows cover the Go API service only, and their post-deploy migration step is non-blocking and does not yet use the dedicated migration command.
Python workers	No CI pipeline. Tests and lint run locally.
Web dashboard	No CI pipeline. Type-check runs as part of the production build.
Release (all three)	A scripted local release builds the three images and tags each with the commit SHA, the semver version and <code>latest</code> . The images can be pushed to a private registry. A polling "GitOps" watcher can rebuild and redeploy a service when its main branch moves.

4.3 Practices visible in the code

- **Specification-driven.** 43 written specifications: 33 with use cases, numbered business rules and NFRs, and 10 lighter improvement specifications. From phase 2 onward, specifications were written alongside
- **Fail loudly.** Ingestion failures are recorded per draft and per document, not swallowed. Extraction parse errors are raised, not returned empty.

or ahead of the code; the phase-1 specifications document code already written.

- **Versioned schema.** 86 paired up/down SQL migrations applied by `golang-migrate`.
- **Standard-library Go.** `net/http` and `database/sql`, no web framework and no ORM. Dependency injection happens in one composition root.
- **Security gates at release.** A cross-project isolation test suite ran with the race detector against a migrated database before v1.0.0 was frozen.
- **Auditability.** Node history is append-only for graph edits (regenerating a document replaces its sections and chunks), and every database write attempt is audited.
- **Evaluation assets.** A merge-precision benchmark and gates for entity resolution, a retrieval evaluation script, and an NL-query benchmark schema.
- **Cost controls in code.** Per-run and per-document LLM cost ceilings, content-hash deduplication, and local models for embedding and re-ranking.

5 Known limitations

Open items a technical reviewer should know about. None blocks the pre-production uses in §1; the §5.3 items are scheduled before any production tenant.

■ 5.1 Processing and capability limits

Area	Current behaviour
Worker job delivery	Queue consumption is at-most-once. A job interrupted by a worker restart is not redelivered automatically. A recovery sweep for stalled entity-extraction runs is implemented but not yet scheduled (PLANNED). Failures inside a job are recorded per item.
PDF length	At most the first 50 pages of a PDF are extracted.
Office documents	DOCX files yield paragraph text only; tables are not extracted. Legacy <code>.doc</code> and <code>.xls</code> , PowerPoint (<code>.pptx</code>) and image files are not accepted.
Structured files	JSON, CSV and XLSX are stored at document level. They are not chunked or embedded. One financial-statement layout is parsed deterministically.
Document-level extraction	The per-document LLM summary step reads the first 12,000 characters. Chunk-level extraction covers the whole text.
Code knowledge graph	Symbols and containment only. No call or import edges, and no embeddings for code symbols.
Embedding throughput	Local embedding runs on CPU and is not parallelised across worker jobs. Large ingests are serialised on it.
Database write path	Supported for PostgreSQL sources only.
Model cache in the release bundle	The release bundle has no persistent model-cache volume, so embedding models are downloaded again when a container is recreated.

■ 5.2 Engineering quality gaps

- The web dashboard has no automated tests and no CI. Outstanding lint findings are not enforced.
- Python has no CI, and coverage measurement is not configured. One unit test fails at the baseline.
- Two advertised MCP tools have no server route. The gateway's contract tests use mock servers, so they do not catch missing routes.

■ 5.3 Hardening items before production

#	Item	Target state
1	Secret management	Wire the in-application cloud secrets provider (a stub today) and move the encryption key to a managed key service, held by as few services as possible.
2	Transport security	The agent gateway requires TLS by default on non-loopback interfaces (today it only warns), with a reference TLS reverse-proxy configuration.
3	Rate limiting	API-wide per-key and per-IP rate limiting in the core server, including login, with state shared across replicas. Today only the agent gateway (per identity, in a single process) and the public ingestion endpoint rate-limit.
4	Authorization hardening	Project scope is enforced today by middleware and on project-scoped handlers. Target: the same per-project role and ownership check on every project- and data-source-scoped operation and every read by ID, including for service and agent keys that are not linked to a user.
5	Dashboard proxy	Every proxied API call requires an authenticated dashboard session, enforced centrally.
6	Internal service authentication	Authenticate the worker service's internal endpoints instead of relying on network isolation.
7	Real-time channels	Restrict WebSocket origins to configured hosts and authenticate the progress streams.

#	Item	Target state
8	Release data hygiene	Move development seed data, including seeded credentials, out of the migrations shipped in the release bundle, and require operator-generated credentials at install time.
9	Network exposure	The release bundle publishes no host ports for the database, cache and worker services, and the cache requires authentication. Only the dashboard and the agent gateway are exposed, behind TLS.
10	Durable job processing	Acknowledged delivery, bounded retries and a dead-letter queue for worker jobs, and a scheduled recovery sweep for stalled extraction runs.
11	SQL validation hardening	Database-level read-only sessions for every supported engine; today only PostgreSQL sources on the agent/API path have one. The dashboard chat path, where the worker service connects to source databases directly, gains parser-based validation, read-only sessions and server-side statement timeouts. Least-privilege credential guidance for connected databases.
12	Cloud delivery	Use the dedicated migration command in the deploy workflows, add CI and container-service delivery for the Python and dashboard services, and bring the managed Redis under infrastructure-as-code.
13	Multi-replica readiness	Coordinate background workers across API replicas, align HTTP timeouts with the long-poll and streaming endpoints, make the indexer readiness check fail when a dependency is down, and define self-hosted backup, restore and RPO/RTO.
14	Security headers and external review	A content security policy and standard security headers on the dashboard, then an independent penetration test before the first production tenant: TO BE COMPLETED
15	Uniform model-call accounting	Route the direct extraction endpoint and the dashboard chat model calls through the same usage log, budgets and circuit breaker as the server's AI layer.

6 Roadmap (proposed)

This roadmap is proposed and is not yet a committed plan. It follows from the limitations in §5 and from the specifications that are not yet delivered (§2). No dates or effort estimates are stated; sequencing and staffing are **TO BE COMPLETED**.

6.1 Now – production readiness

Workstream	Scope
Hardening before production	Items 1–15 in §5.3. Items 10 and 12 are expanded in the next two rows.
Durable job processing	§5.3 item 10: acknowledged delivery with a visibility timeout, bounded retries and a dead-letter queue for every worker queue (for example Redis Streams consumer groups).
CI for every repository	Python: tests, lint and coverage in CI; fix the one failing test. Web: a component and end-to-end test suite (critical flows: login, ingestion, chat, data sources), lint gating in CI.
Release pipeline for all images	§5.3 item 12: registry and deploy automation for the Python and dashboard images, matching the Go service. Git tags for releases.
Dependency licence posture	Pin the cache service to a permissively licensed version or a drop-in alternative (§7.4). Add a licence check to CI.
Gateway completeness	Implement or withdraw the two MCP tools that have no server route. Add a contract test against the real router.

6.2 Next – capability depth

Workstream	Scope
Accuracy measurement	Put the built benchmark suite (BA-013) to work on NL-to-SQL, and report answer accuracy separately from request completion.

Smart context (BA-020)	Populate table embeddings, preferably with the local embedding stack, so that schema selection scales to large databases.
Richer document coverage	DOCX tables, configurable PDF page limits, chunking and embedding for tabular files, more office formats.
Code knowledge (BA-003/026)	Call and import edges, code-symbol embeddings, and first use on real repositories.
Entity-resolution operations	A dashboard trigger and a review queue for merge suggestions. Automatic extraction on ingest, subject to the cost ceilings.
Embedding throughput	Parallel or off-loop embedding, with optional GPU inference.

6.3 Later – breadth and scale-out

Workstream	Scope
Connectors	MongoDB (IMP-003); database write path for engines other than PostgreSQL.
Retrieval	Graph community detection and summaries (the remaining BA-033 scope).
Scale-out	Distributed rate limiting for gateway replicas; managed cloud queue; horizontal worker scaling.
Pilot deployments	TO BE COMPLETED

7 IP & dependencies

7.1 First-party code

All code in the three repositories (Go API and MCP gateway, Python workers, web dashboard) and all specifications in the requirements repository are first-party and proprietary. None of it is published under an open-source licence. The web package is marked private.

Item	State
Owning legal entity	TO BE COMPLETED
Contributor IP assignment	TO BE COMPLETED
Copyright / licence notices in repositories	None of the repositories has a licence or notice file yet. Adding a proprietary notice is a housekeeping item.
Third-party code provenance	A keyword scan for provenance comments found one design-idea acknowledgement (a graph-layout seeding approach) and no markers of vendored third-party source. A snippet-level provenance scan has not been performed.

7.2 Licence audit summary (third-party runtime components)

Method: offline reads of licence files and package metadata already on disk. Go licences come from the module cache, for every module linked into the three server binaries. Python licences come from installed package metadata, for the runtime set exported from the lockfile with development tools excluded. Web licences come from installed `package.json` files, for the full production dependency tree. Full tables are in §8.

Ecosystem	Packages audited	Licence mix	Flags
Go (server, gateway, migrator)	46 modules (16 direct)	Apache-2.0 18 · MIT 12 (+2 MIT/Apache dual) · BSD 11 · MPL-2.0 3	MPL-2.0 (weak, file-level): MySQL driver and 2 small error-handling utilities

Ecosystem	Packages audited	Licence mix	Flags
Python (workers)	87 packages (27 direct)	Predominantly MIT, BSD and Apache-2.0; also ISC, PSF and MPL-2.0	LGPL-2.1: SQL Server driver (pymssql). MPL-2.0: certifi , pathspec , tqdm (in part)
Web (dashboard)	752 packages (34 direct)	MIT 653 · ISC 49 · BSD 24 · Apache-2.0 14 · other permissive 10	LGPL-3.0: native image library pulled in by the framework's image optimiser. CC-BY-4.0: one browser-support data package

Copyleft assessment

No GPL or AGPL component was found in the audited runtime sets. The weak-copyleft components are used unmodified, through their normal library interfaces:

- The **MPL-2.0** obligations are file-level and apply only if those files are modified.
- The **LGPL** components are a Python extension module loaded dynamically, and a shared native library that the web framework loads for image optimisation.

Distributing container images that contain LGPL components carries notice and relinking obligations. Counsel should confirm this before distribution.

Scope limits: operating-system packages in the container base images were not audited. That includes Debian and Alpine packages and the OCR engine's system dependencies. The code-parser language pack bundles grammars under their own licences, which were not audited one by one. Two packages installed in the local Python environment outside the lockfile were excluded; one of them is LGPL-3.0, and neither ships in the image.

7.3 Models and engines

Component	Role	Licence	Verification
multilingual-e5-small (384-dim)	Embeddings for chunks, sections, entities, memory	MIT	Verified from the locally cached model card
bge-reranker-v2-m3	Cross-encoder filter in entity resolution	Apache-2.0	Per the publisher's model card; not verified offline
Tesseract OCR 5 (+ Vietnamese data)	Page OCR	Apache-2.0	Engine version confirmed in the worker image
RapidOCR (ONNX runtime)	Structured-figure OCR	Apache-2.0	Package metadata
tree-sitter + grammars	Code parsing	MIT	Package metadata (grammar pack: see §7.2)
PyTorch (CPU build)	Model inference runtime	BSD-3-Clause	Package metadata

7.4 External services and vendor risk

Dependency	Licence / terms	Risk and mitigation
LLM providers	Commercial API terms of the chosen provider	Pluggable: providers are registered at runtime, with priority failover and per-function model routing. Both Anthropic-native and OpenAI-compatible endpoints are supported, including self-hosted open-weight models. There is no hard dependency on one vendor. Data-processing terms depend on the deployment: TO BE COMPLETED
PostgreSQL 16 + pgvector 0.8	PostgreSQL Licence (permissive), both	Standard open-source database; available as a managed service in every major cloud.

Dependency	Licence / terms	Risk and mitigation
Redis (queues, short-lived state)	Redis ≤ 7.2: BSD-3-Clause. Redis 7.4.x: RSALv2 / SSPLv1 dual licence. Redis 8+: adds an AGPLv3 option.	Both Compose files reference the floating <code>redis:7-alpine</code> tag, and the running development image resolves to 7.4.8 , which falls under the RSALv2/SSPLv1 terms. DAAB uses Redis only as an internal queue and does not offer it as a service. Mitigation: pin <code>7.2.x</code> or switch to a BSD-licensed drop-in fork such as Valkey. The managed cloud cache already runs outside the bundle. Counsel should confirm; this is not a legal opinion.
Optional identity provider	Supabase Auth (hosted service)	Optional single sign-on. Username/password login works without it.
Optional embedding API	Provider API terms	Used only for the optional table-selection feature (§1.2, BUILT). Core embeddings run locally.

8 Appendix – licence tables

8.1 Go – direct dependencies (versions from `go.mod`)

Module	Version	Licence	Use
<code>github.com/aws/aws-sdk-go-v2</code> (+ config, cloudwatch, cloudwatchlogs)	1.32.7	Apache-2.0	Optional cloud logs and metrics
<code>github.com/go-sql-driver/mysql</code>	1.9.3	MPL-2.0	MySQL / MariaDB driver
<code>github.com/golang-migrate/migrate/v4</code>	4.18.1	MIT	Schema migrations
<code>github.com/google/jsonschema-go</code>	0.4.3	MIT	MCP tool schemas
<code>github.com/google/uuid</code>	1.6.0	BSD-3-Clause	Identifiers
<code>github.com/gorilla/websocket</code>	1.5.3	BSD-2-Clause	Progress channel
<code>github.com/lestrrat-go/jwx/v2</code>	2.1.3	MIT	Optional SSO token verification
<code>github.com/lib/pq</code>	1.10.9	MIT	PostgreSQL driver
<code>github.com/microsoft/go-mssqldb</code>	1.7.2	BSD-3-Clause	SQL Server driver
<code>github.com/modelcontextprotocol/go-sdk</code>	1.6.1	MIT → Apache-2.0 (transition)	MCP server
<code>golang.org/x/crypto</code> , <code>golang.org/x/time</code>	0.48 / 0.15	BSD-3-Clause	bcrypt; rate limiting
<code>gopkg.in/yaml.v3</code>	3.0.1	MIT + Apache-2.0	Configuration
<code>github.com/stretchr/testify</code>	1.9.0	MIT	Tests only (not linked)

Transitive MPL-2.0 modules: `hashicorp/errwrap` 1.1.0 and `hashicorp/go-multierror` 1.1.1. All other transitive modules (AWS SDK internals, `smithy-go`, `jmespath`, `lestrrat-go` helpers, `segmentio` encoders, `golang.org/x/{oauth2,sys,text}`, and others) are Apache-2.0, MIT or BSD.

8.2 Python – direct runtime dependencies (installed versions)

Package	Version	Licence	Package	Version	Licence
<code>fastapi</code>	0.135.2	MIT	<code>pymssql</code>	2.3.13	LGPL-2.1
<code>uvicorn</code>	0.42.0	BSD-3-Clause	<code>pymysql</code>	1.2.0	MIT
<code>httpx</code>	0.28.1	BSD-3-Clause	<code>asyncpg</code>	0.31.0	Apache-2.0
<code>pydantic / pydantic-settings</code>	2.12 / 2.13	MIT	<code>cryptography</code>	48.0.0	Apache-2.0 OR BSD-3
<code>redis (client)</code>	7.4.0	MIT	<code>pypdf</code>	6.13.2	BSD-3-Clause
<code>anthropic</code>	0.86.0	MIT	<code>pypdfium2</code>	5.10.1	BSD-3 / Apache-2.0
<code>sentence-transformers</code>	5.4.1	Apache-2.0	<code>pytesseract</code>	0.3.13	Apache-2.0

Package	Version	Licence	Package	Version	Licence
torch (CPU)	2.12.0	BSD-3-Clause	rapidocr-onnxruntime	1.4.4	Apache-2.0
numpy	2.4.4	BSD-3 (+ permissive)	Pillow	12.2.0	MIT-CMU
tree-sitter (+ TS, Python, Go)	0.25.2	MIT	python-docx	1.2.0	MIT
tree-sitter-language-pack	1.8.1	MIT (bundled grammars vary)	openpyxl	3.1.5	MIT
pathspec	1.0.4	MPL-2.0	transformers / huggingface-hub	5.5 / 1.10	Apache-2.0

Transitive runtime packages flagged: `certifi` (MPL-2.0), `tqdm` (MPL-2.0 AND MIT), `regex` (Apache-2.0 AND CNRI-Python), `typing_extensions` (PSF-2.0). The remaining transitive packages (`scipy`, `scikit-learn`, `onnxruntime`, `opencv-python`, `shapely`, `tokenizers`, `safetensors`, `starlette`, `jinja2`, `lxml`, `protobuf`, and others) are MIT, BSD, ISC or Apache-2.0.

■ 8.3 Web – production dependencies (installed versions)

Licence	Count	Direct dependencies under this licence
MIT	653	next 16.2.1, react / react-dom 19.2.4, @tanstack/react-query, iron-session, @supabase/supabase-js, three, @react-three/fiber · drei · postprocessing, 3d-force-graph, react-force-graph(-2d/-3d), cytoscape (+ cola, dagre), recharts, framer-motion, react-markdown, remark-gfm, rehype-highlight, @base-ui/react, cmdk, clsx, tailwind-merge, tw-animate-css, shadcn, type packages
ISC	49	lucide-react
BSD-2 / BSD-3	24	(transitive only)
Apache-2.0	14	class-variance-authority
Zlib · Unlicense · 0BSD · BlueOak-1.0.0 · Python-2.0 · dual permissive · MIT (unlabelled manifest)	10	postprocessing (Zlib), ogl (Unlicense)
LGPL-3.0-or-later	1	(transitive) native image library used by the framework's image optimiser
CC-BY-4.0	1	(transitive) browser-support data table

One transitive package has no licence field in its manifest; its bundled licence file is MIT. The native image library is installed in a platform-specific build, so the build inside the Linux container image differs from the one audited on the development machine. Both carry the same licence.

■ 8.4 Method and reproducibility

- **Go:** `go list -deps` over the three command packages to find linked modules; licence text classified from each module's LICENSE file in the local module cache (no network).
- **Python:** `uv export --frozen --no-dev` for the runtime set; `License-Expression`, `License` and classifier fields read from installed `METADATA`.
- **Web:** `npm ls --omit=dev --all` for the production tree; the `license` field read from each installed `package.json`.
- **Services:** image tags read from the Compose files; the running Redis and pgvector versions read from the development containers.